

Uber Architecture

Ride-Sharing System Design - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	What is Uber? High-Level Flow	What happens when I book a ride?
2	Capacity Estimation	How big are traffic, location data and storage?
3	Layer 1: Client and Edge (Ramen, API Gateway)	How do phones stay connected safely?
4	Layer 2: Real-Time Location Tracking	How are millions of GPS pings handled?
5	Layer 3: Geospatial Indexing with H3	How do we find nearby drivers instantly?
6	Layer 4: DISCO Matching Engine	How is the best driver chosen, without double booking?
7	Layer 5: Storage and Persistence	Where does each kind of data live?
8	End-to-End Walkthrough	How do all pieces work together?
9	Cheat Sheet and Interview Q&A	What must I remember?

1. What is Uber? High-Level Architecture

Uber is a **ride-sharing app**. The rider app sends a ride request; the backend finds nearby drivers; a **matching algorithm selects the best driver**; the driver is notified; the driver accepts; and the ride is confirmed. This is the **ride booking life cycle**, the basic high-level architecture.

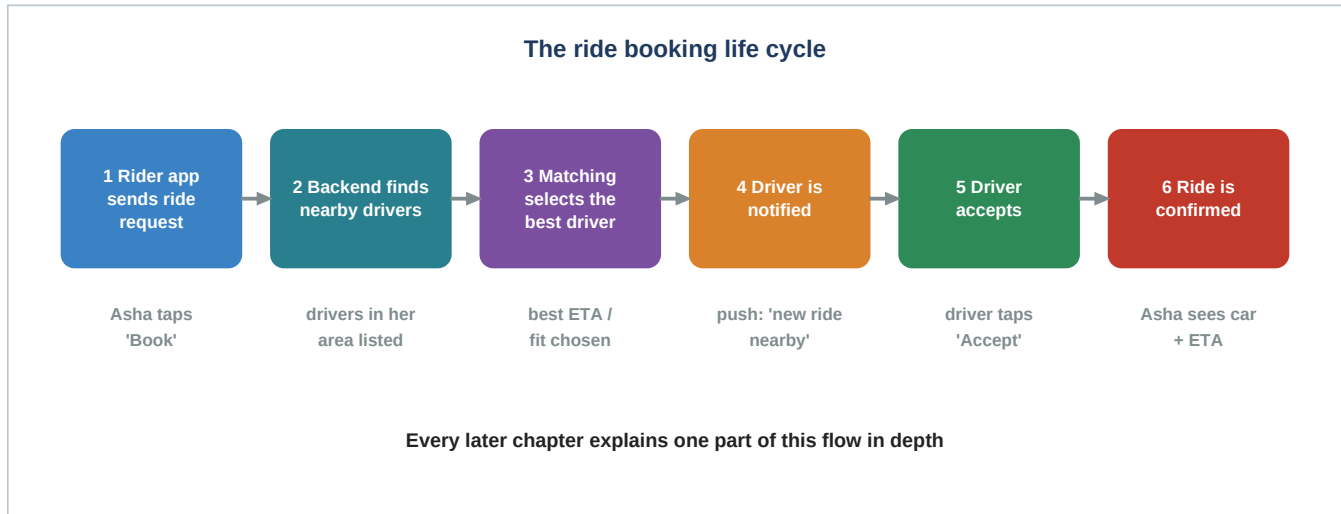


Figure 1: The ride booking life cycle

EXAMPLE

Asha opens the app in Delhi and taps 'Book'. The system looks at drivers around her, picks the one who can reach her fastest, pings that driver's phone, the driver taps 'Accept', and Asha's screen switches to 'Driver Ravi is 4 minutes away'.

1.1 What makes it hard

- **Real-time data at huge scale:** millions of drivers continuously report GPS positions.
- **Fast spatial search:** 'who is near this rider?' must be answered in milliseconds, not by scanning every driver.
- **Correctness:** a driver must never be assigned to two riders at once; payments must be exact.
- **Resilience:** messages must not get lost when a service is busy.

The five layers of the Uber architecture

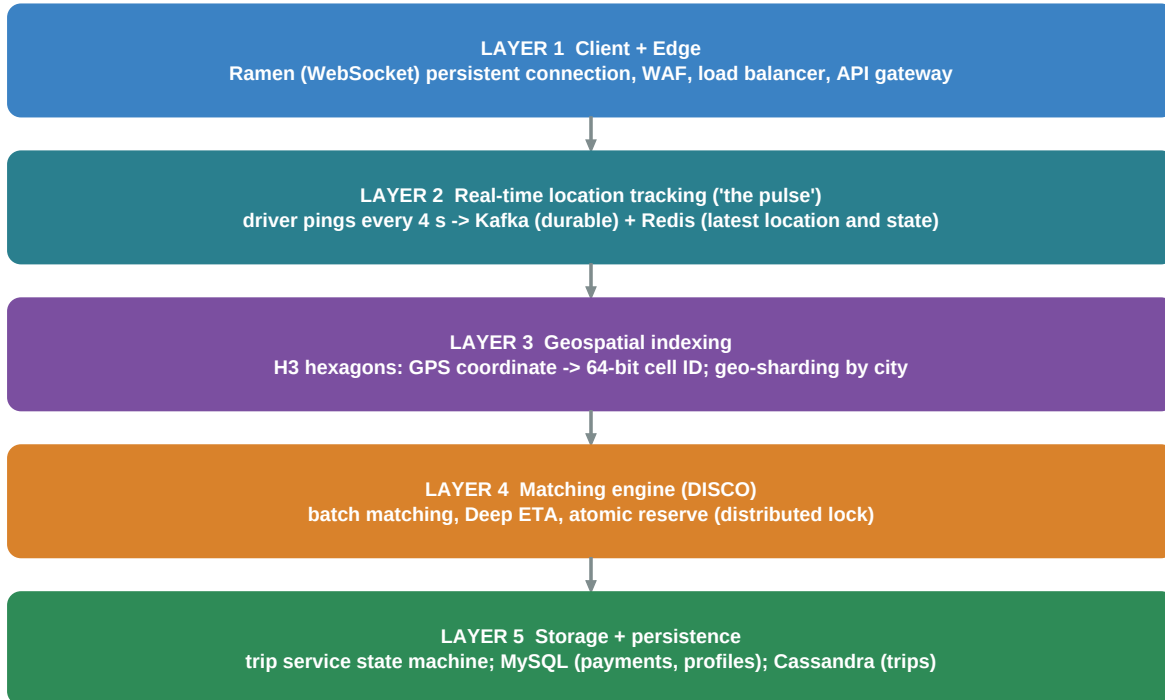


Figure 2: The five layers, which the rest of these notes explain one by one

2. Capacity Estimation

Before designing, estimate **traffic**, **data size** and **storage** so each component can be sized properly. The numbers below are the assumptions used in the video.

2.1 Traffic and scaling

- **Monthly active users (MAU):** assume about **200 million**.
- **Daily active users (DAU):** about **40 million**, and about **40 million daily trips**.
- **Active drivers:** about **2 million**.
- **Average ride requests per second** = $40,000,000 / 86,400$ seconds per day = about **460 requests/s**.

EXAMPLE

460 ride requests/s is tiny compared with the location stream below. Booking requests are not the bottleneck: **location updates** are.

2.2 Location update capacity (the most demanding part)

The system tracks real-time GPS, so it needs **real-time GPS streaming**. Each active driver sends a location update **every 4 seconds**.

- $2,000,000 \text{ drivers} / 4 \text{ s} = \mathbf{500,000 \text{ writes per second}}$.

Payload size per update

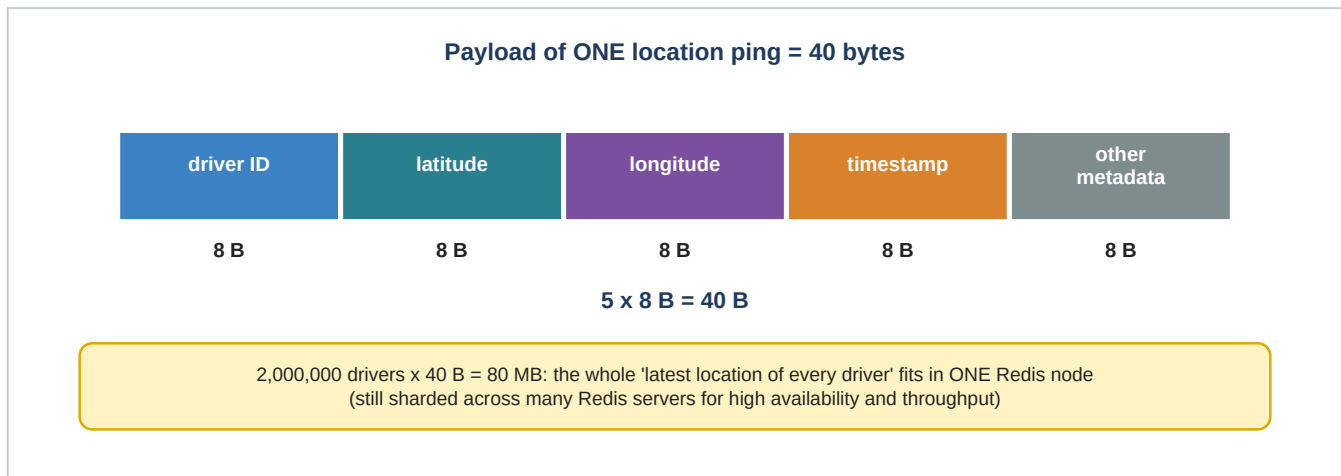


Figure 3: A location ping is 5 fields of 8 bytes = 40 bytes

- driver ID (8 B) + latitude (8 B) + longitude (8 B) + timestamp (8 B) + other metadata (8 B) = **40 bytes**.
- One 'round' of updates from all drivers: 2,000,000 x 40 B = **80 MB**.
- That is small enough to **fit in a single Redis node**; in practice it is spread over many Redis servers (sharded) for **high availability and throughput**.

EXAMPLE

Driver 101's ping: id=101, lat=28.6139, lng=77.2090, t=10:00:04, metadata. Eight bytes each for five fields: 40 bytes in total, roughly the size of a short text message but sent 15 times a minute by every driver.

2.3 Storage estimation

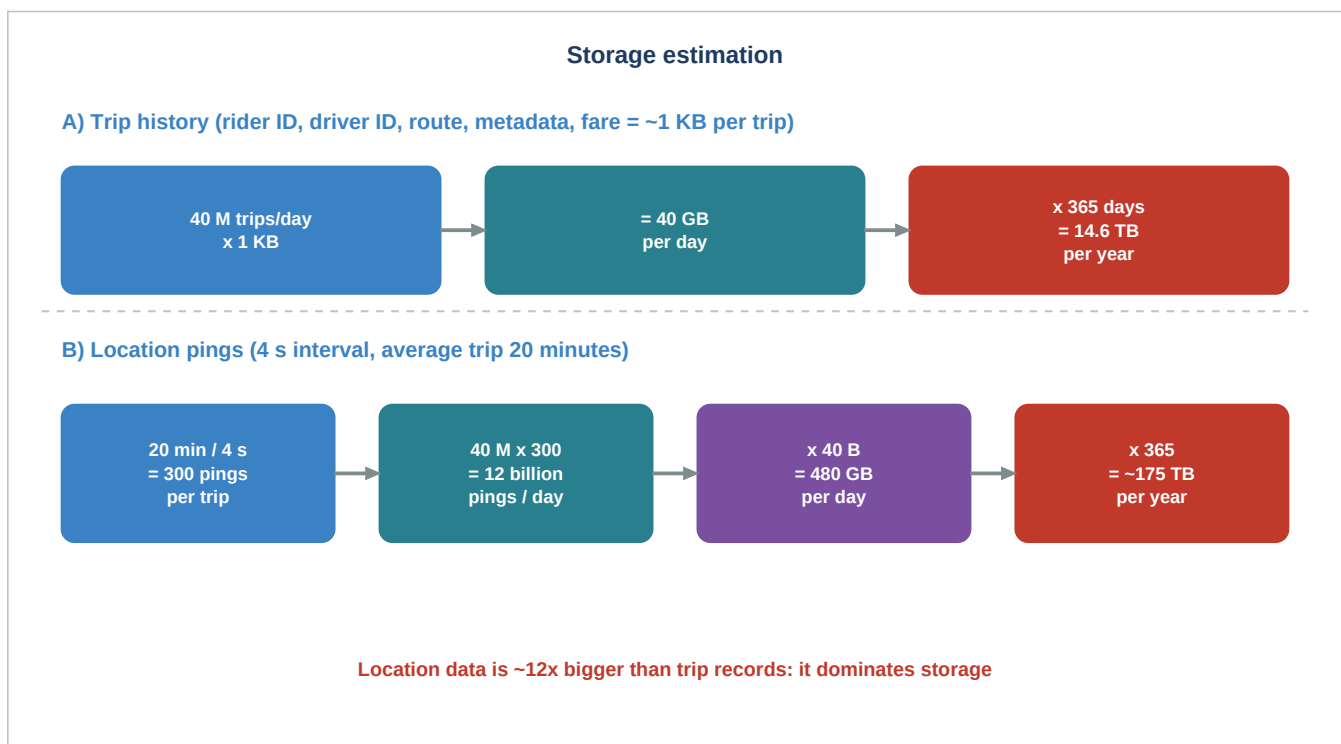


Figure 4: Trip records are small; location pings dominate storage

Trip history (transactions)

- A trip record holds rider ID, driver ID, route, metadata and fare: about **1 KB**.
- 40 M trips/day x 1 KB = **40 GB per day**; x 365 = **14.6 TB per year**.

Location pings

- Average trip lasts 20 minutes: $20 \times 60 / 4 \text{ s} = \mathbf{300 \text{ pings per trip}}$.
- Daily pings: 40 M trips x 300 = **12 billion** events per day.
- Daily storage: 12 B x 40 B = **480 GB per day**; yearly about **175 TB**.

EXAMPLE

Why store pings at all? For safety investigations, billing disputes (was the route really that long?) and machine learning (training better ETA models). That is why Uber keeps user profile, location and billing data for every trip.

2.4 Which database for how long

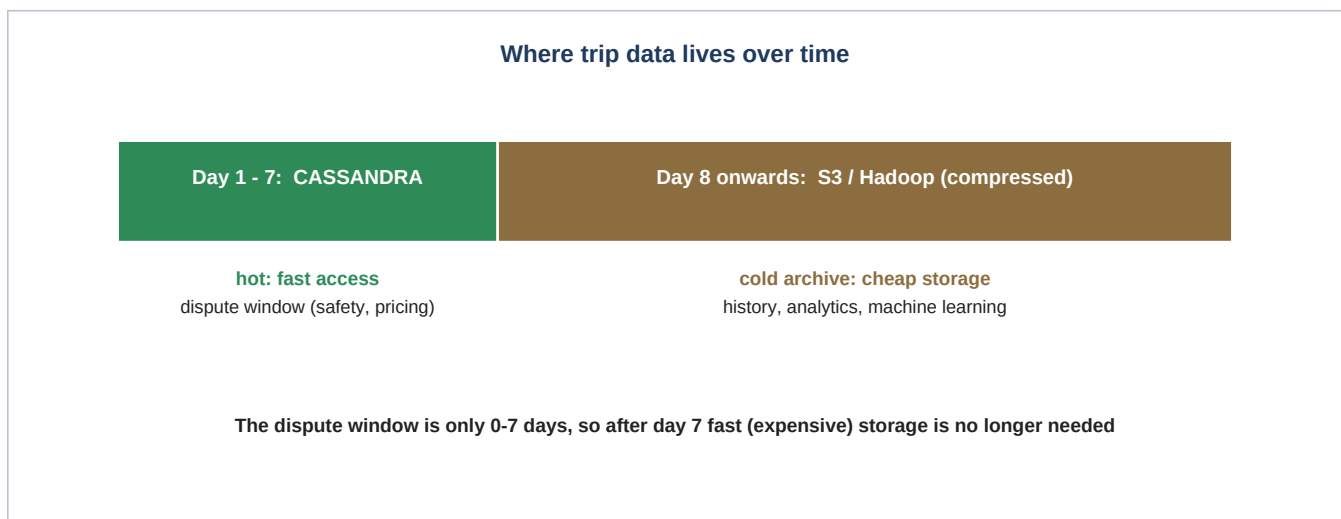


Figure 5: Hot storage for the dispute window, cold archive afterwards

- **Days 1 to 7:** trip data stays in **Cassandra** for fast access to recent disputes, safety issues or pricing questions.
- **Day 8 onwards:** moved to **cold storage** such as **Hadoop or S3** in compressed form; the dispute window is over.

EXAMPLE

A rider disputes a fare 3 days after the trip: support reads the route from Cassandra instantly. A data scientist wants last year's trips: they read compressed files from the archive, slower but far cheaper.

2.5 Network bandwidth

- **Incoming:** 500,000 pings/s x 40 B = 20,000,000 B/s = **20 MB/s** (about 160 megabits/s).
- **Outgoing (broadcast to riders):** if 5 riders watch one driver on their map, outgoing traffic is about **5x** the incoming: roughly **100 MB/s** (about 800 megabits/s).

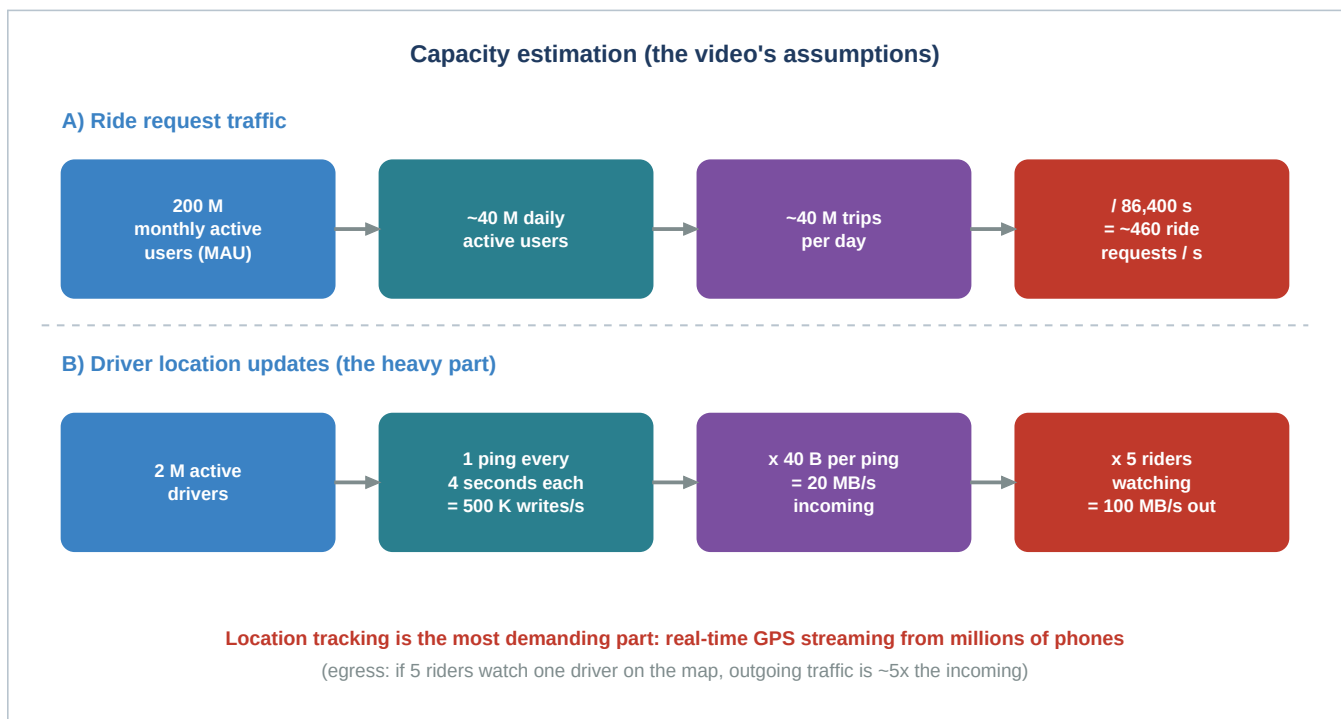


Figure 6: From users to requests per second and from drivers to bandwidth

UNITS NOTE

The video quotes the bandwidth as '20 megabit per second' and '100 Mbps'. Multiplying 500,000 x 40 bytes gives 20 **megabytes** per second, which is about 160 megabits per second. The notes use megabytes and give the megabit equivalent. The relative picture (egress is about 5x ingress) is unchanged.

3. Layer 1: Client and Edge

3.1 Ramen: a persistent connection (Uber's WebSocket-style platform)

Unlike a standard app that uses ordinary HTTP request/response, Uber uses a **push-based platform** called **Ramen** (Real-time Asynchronous Messaging Network, Uber's own version of the WebSocket idea). It keeps a **live pipe open to every phone to stream GPS pings and trip updates in real time**.

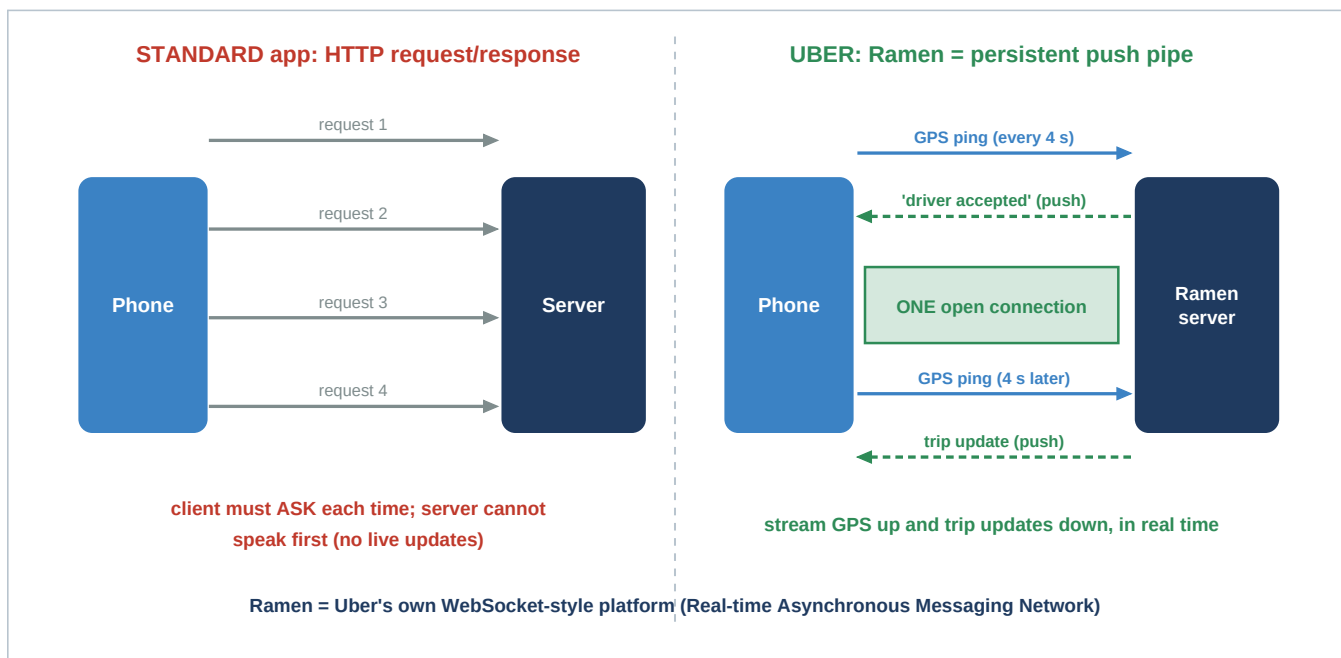


Figure 7: Request/response vs a persistent push pipe

EXAMPLE

With plain HTTP, your app would have to keep asking 'has a driver accepted yet?' every second. With Ramen, the server simply pushes 'Ravi accepted' down the already-open connection the instant it happens, and the driver's phone streams its GPS up the same pipe every 4 seconds.

3.2 API gateways and the edge

The API gateway acts as the **front door**. It handles **authentication**, **rate limiting**, **routing requests to internal microservices** and works with the **firewall (WAF)** and **load balancer**.

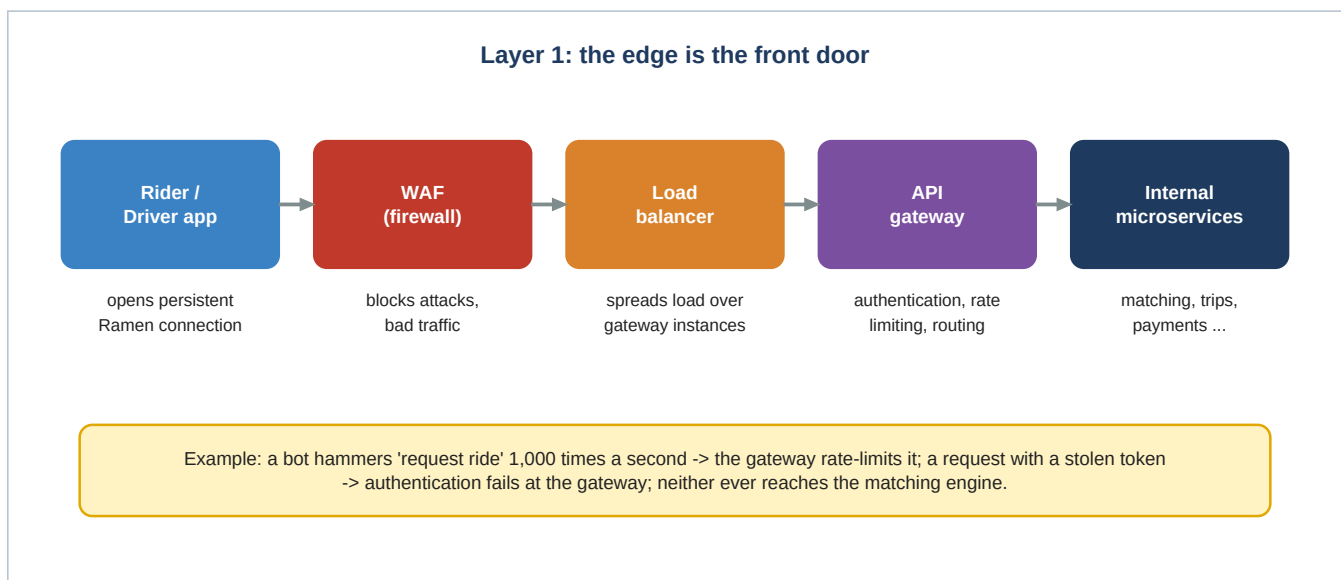


Figure 8: The edge layer in front of the microservices

- **WAF (web application firewall):** blocks malicious traffic.

- **Load balancer:** spreads traffic across gateway instances.
- **API gateway:** checks who you are, limits abuse, forwards to the right service.
- Everything behind it is a set of **microservices** (location, matching, trips, payments...).

EXAMPLE

A script floods 'request ride' thousands of times per second: rate limiting throttles it. A request with an expired login token: authentication rejects it. Neither reaches the matching engine.

4. Layer 2: Real-Time Location Tracking ('the Pulse')

This layer absorbs the **massive stream of pings** that drivers' phones send every 4 seconds over their persistent connection.

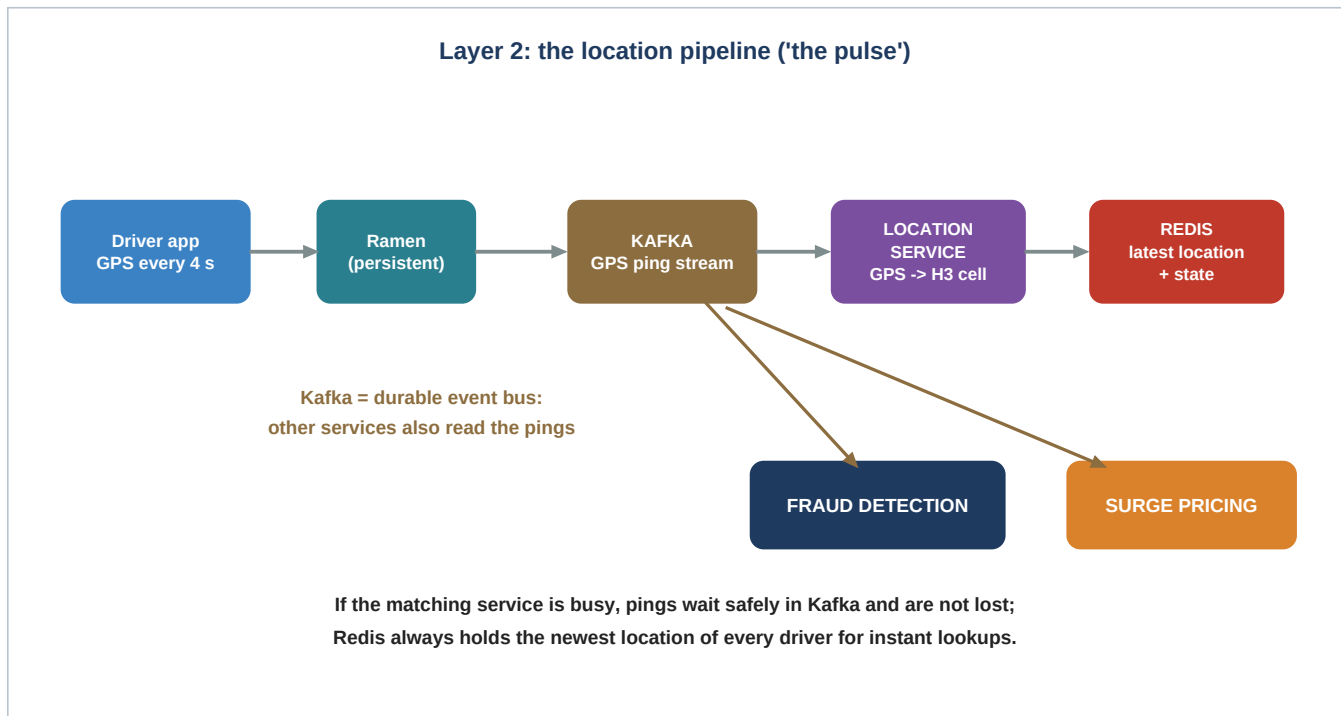


Figure 9: Driver pings flow through Ramen into Kafka, the location service and Redis

4.1 Kafka: durable event bus

All pings are stored in **Kafka**, a highly scalable event bus, so they are **durable and not lost**. If the matching service is busy, the location data simply waits in Kafka.

EXAMPLE

During a rain storm, request volume spikes and the matching service slows down. Pings keep arriving, but instead of being dropped they queue safely in Kafka; services catch up later. Other consumers (fraud detection, surge pricing) read the same stream.

4.2 Redis: latest location and state for fast lookup

The **latest location of every driver** is stored in **Redis**, an in-memory store, for **fast lookups**. It holds the **current state** of all drivers and uses **geospatial sharding** so that queries for a specific city hit the correct server cluster.

Kafka vs Redis in the location layer: different jobs

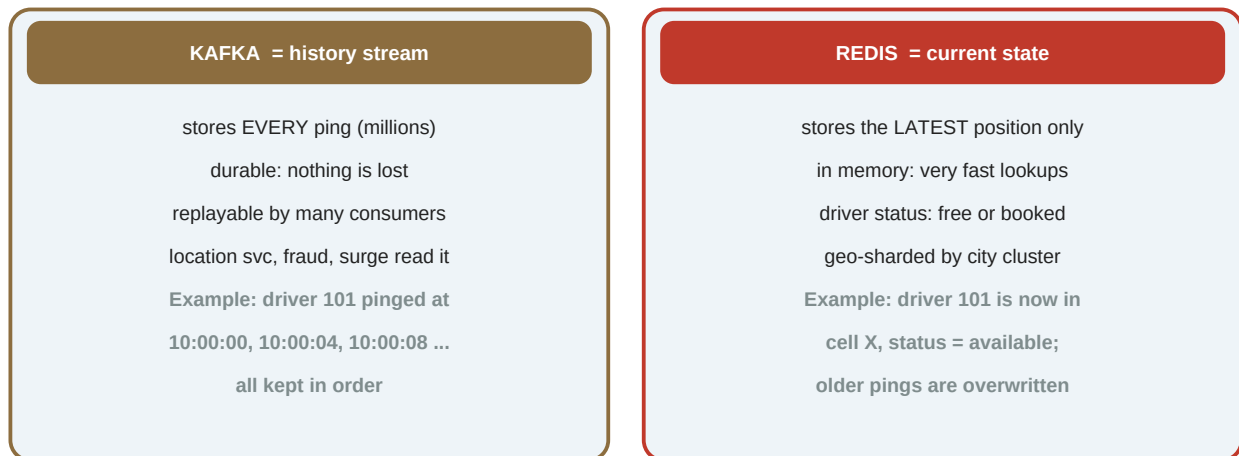


Figure 10: Kafka remembers the history; Redis knows the present

4.3 Driver state

A driver has only two states: **available (free)** or **on trip (booked)**. Only available drivers get new requests.

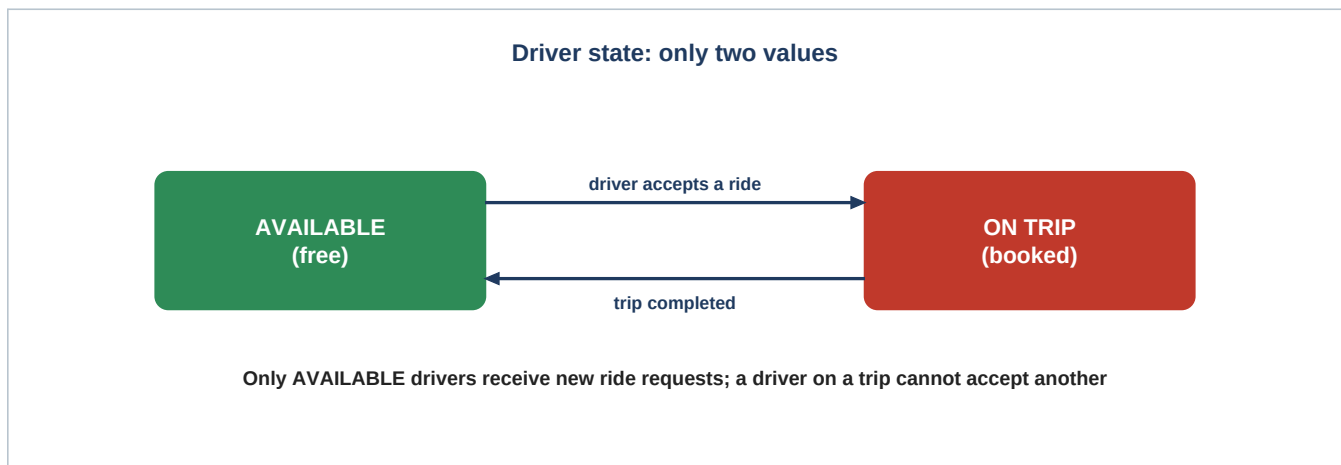


Figure 11: Two-state driver status

EXAMPLE

Driver 245 is mid-trip, so even if she is the closest car to a new rider, she is never offered that ride.

5. Layer 3: Geospatial Indexing with H3

This is described as the most important part of the design: how to find nearby drivers **efficiently**. Uber uses **H3**, its open-source **hexagonal hierarchical spatial index**. It **partitions the entire globe into hexagons** and gives each cell a unique **64-bit ID**. (Google's S2 library is the similar cell-ID idea based on squares-on-a-sphere; the video mixes the two and says 'S3'; here H3 is the Uber hexagon index.)

5.1 Why hexagons?

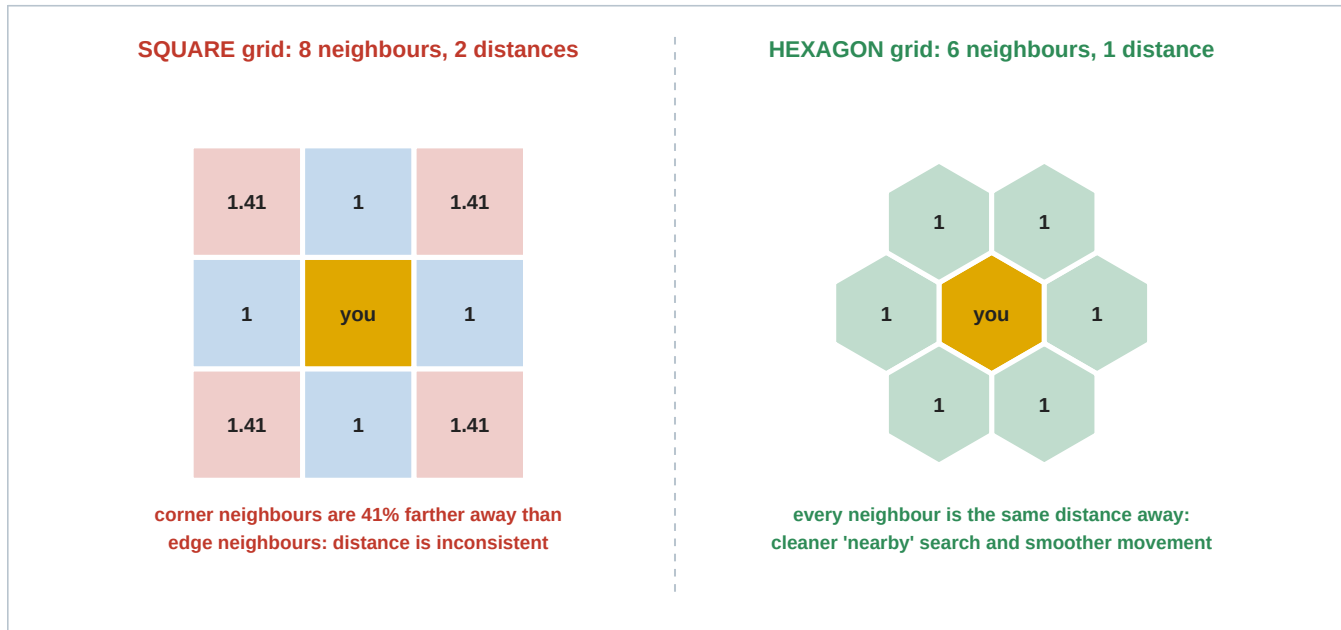


Figure 12: Hexagons have six equidistant neighbours; squares have two kinds of neighbour

EXAMPLE

In a square grid your 8 neighbours are at two different distances (4 touching edges at 1.0, 4 touching corners at about 1.41). In a hexagon grid all 6 neighbours are exactly the same distance. That makes 'search the cells around me' uniform and gives smooth neighbourhood traversal as a driver moves.

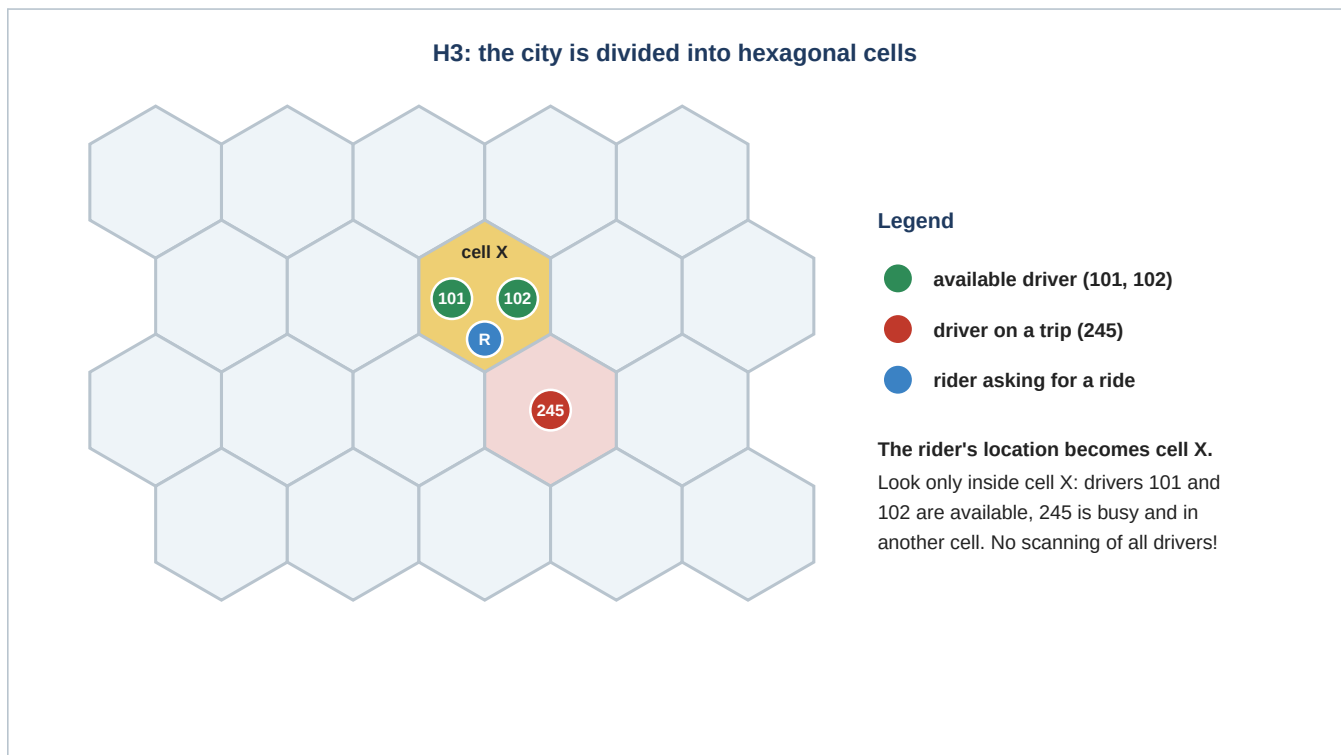


Figure 13: A city divided into hexagonal H3 cells; drivers and the rider are in cells

5.2 Converting a location to a cell ID

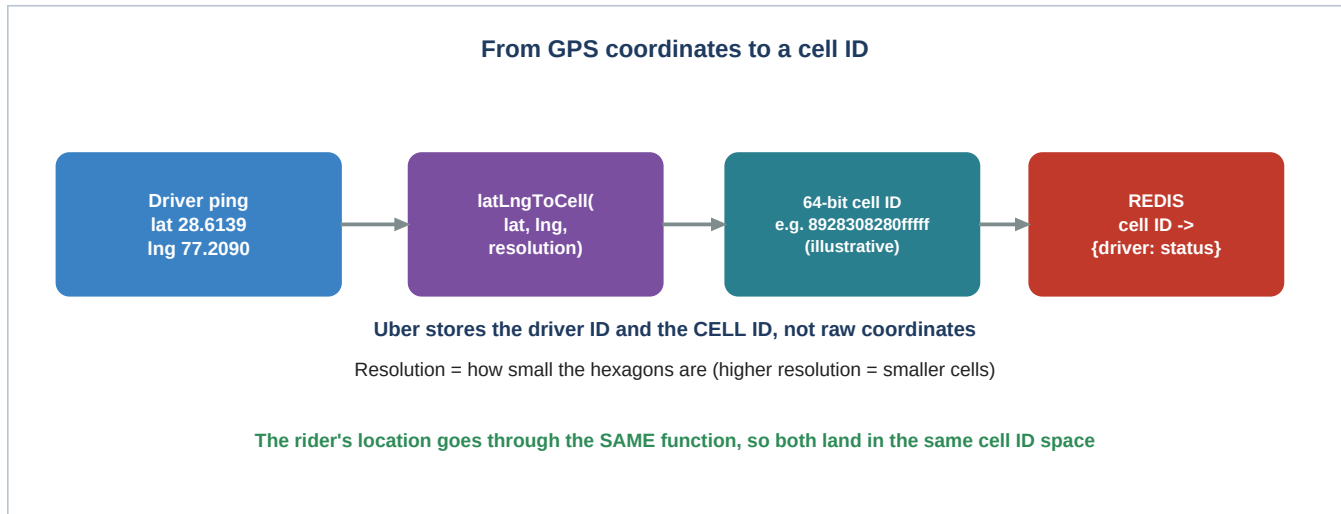


Figure 14: latitude + longitude + resolution -> 64-bit cell ID

- The location service calls a function (conceptually `latLngToCell(lat, lng, resolution)`).
- The **resolution** picks the cell size (higher resolution = smaller hexagons).
- It returns a **64-bit cell ID**.
- **Uber does not store the actual coordinates in this index**; it stores the **driver ID and its cell ID**.

EXAMPLE

Driver ping lat 28.6139, lng 77.2090 -> cell ID 8928308280ffff (illustrative). Redis now records: in cell 8928308280ffff, driver 101 is available.

5.3 The cell table in Redis

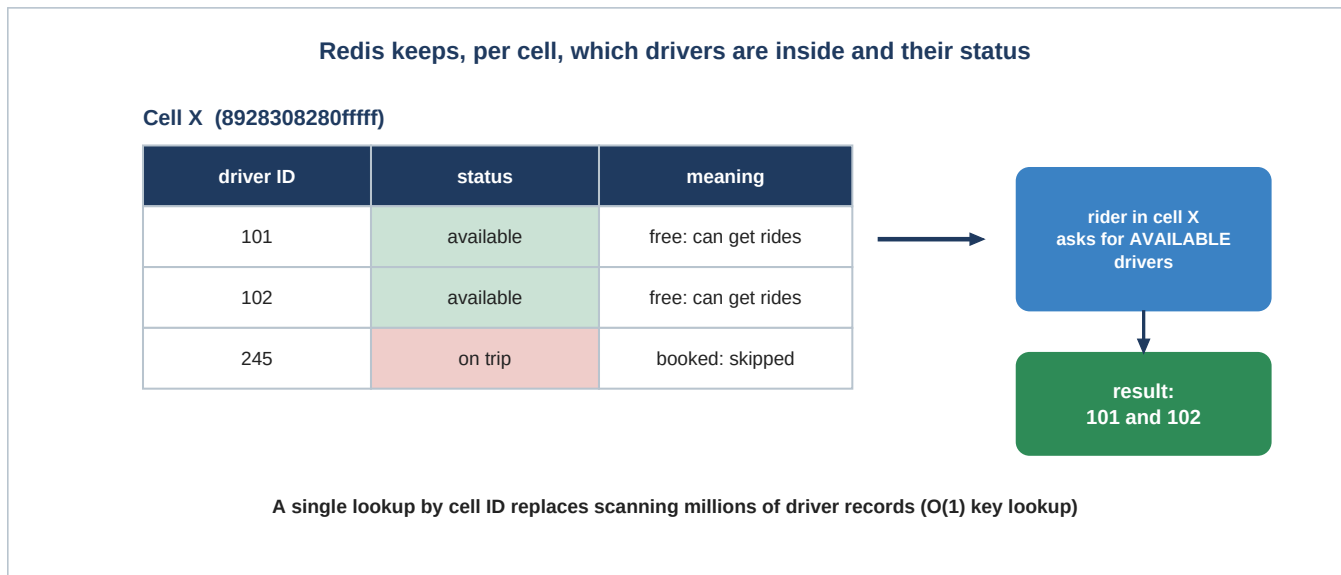


Figure 15: For each cell, Redis stores which drivers are inside and their status

EXAMPLE

Cell X has driver 101 (available), driver 102 (available) and driver 245 (on trip).

5.4 A ride request, step by step

- Rider's latitude/longitude is passed through the **same** function, giving the rider's cell ID (say cell X).
- Fetch the **available** drivers in cell X: the answer is drivers 101 and 102.
- The app/matching engine picks one (say 101) and offers the ride.
- When the driver accepts, the driver's status changes from **available** to **on trip**.

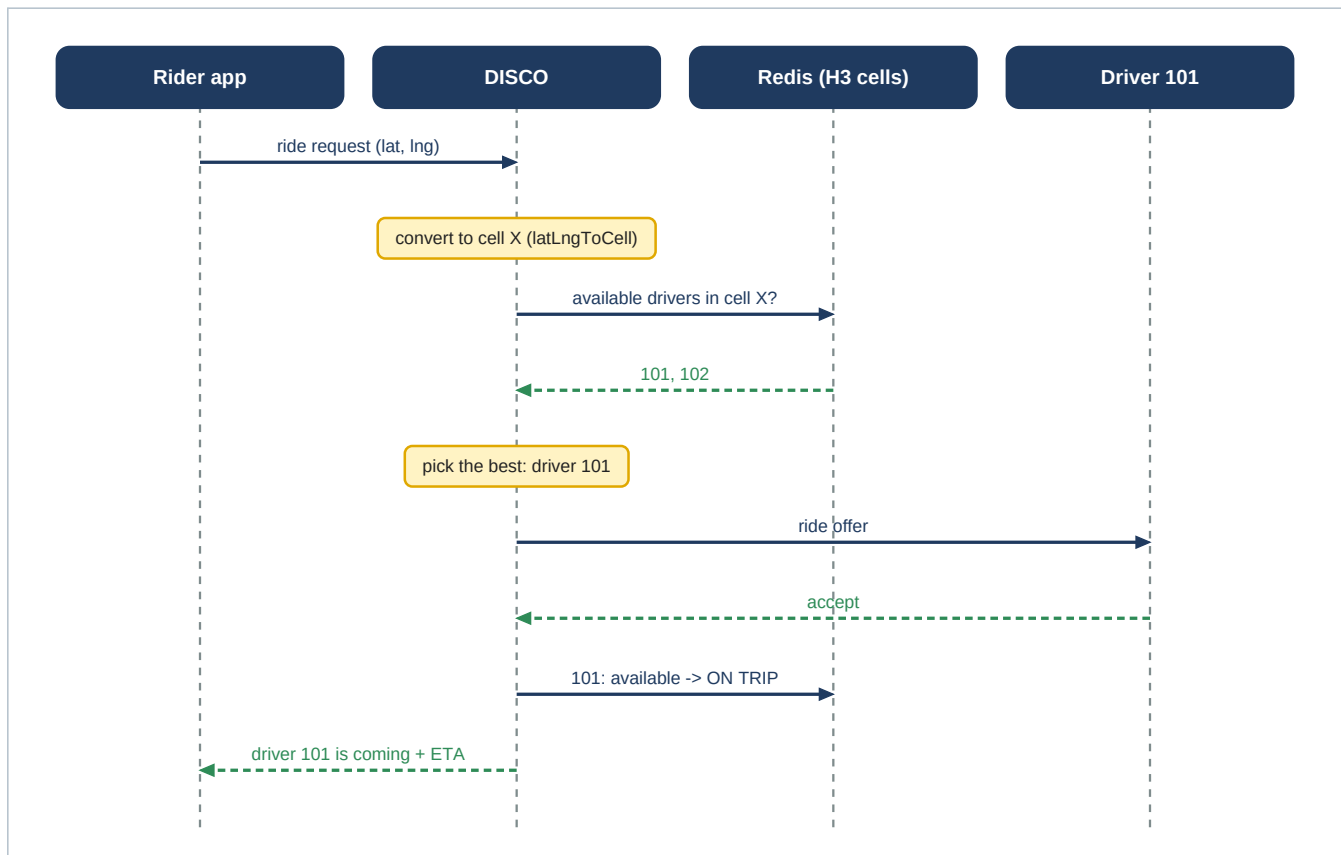


Figure 16: Sequence of a ride request using cell lookups

EXAMPLE

Notice the efficiency: instead of computing the distance to 2 million drivers, Uber looks up one cell ID in Redis and gets a short candidate list in one lookup (a constant-time key lookup).

5.5 How locations update as a driver moves

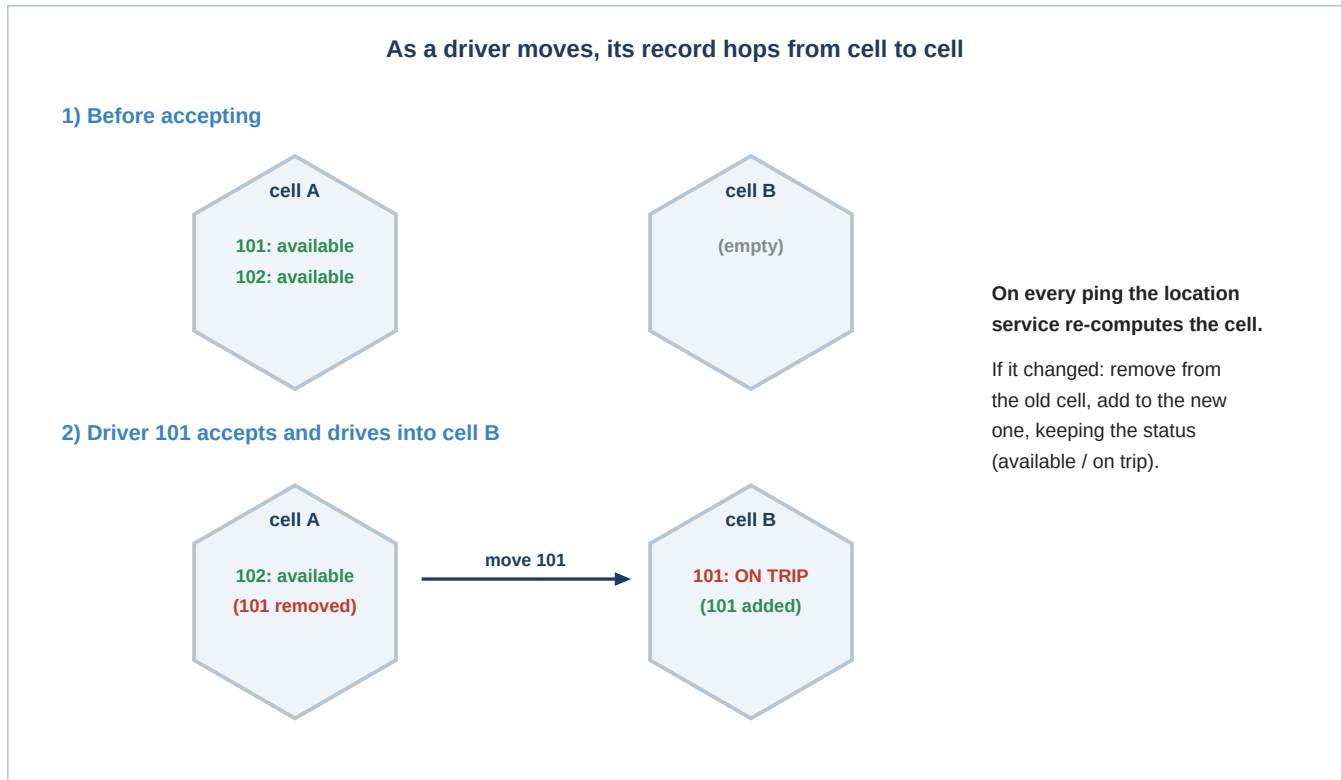


Figure 17: A moving driver's record is removed from the old cell and added to the new one

- On every ping the location service recomputes the driver's cell.
- If the cell changed, the driver's entry is **removed from the old cell and added to the new cell**, keeping the status.
- A driver on a trip keeps moving between cells with status **on trip**, so they are never offered another ride.

5.6 No driver in the cell? K-ring search

If your hexagon has no available driver, the system searches **neighbouring hexagons**. This is called a **K-ring search**: first your cell ($k = 0$), then the ring of touching cells ($k = 1$), then the next ring ($k = 2$), and so on up to the chosen K , stopping at the first ring where drivers are found.

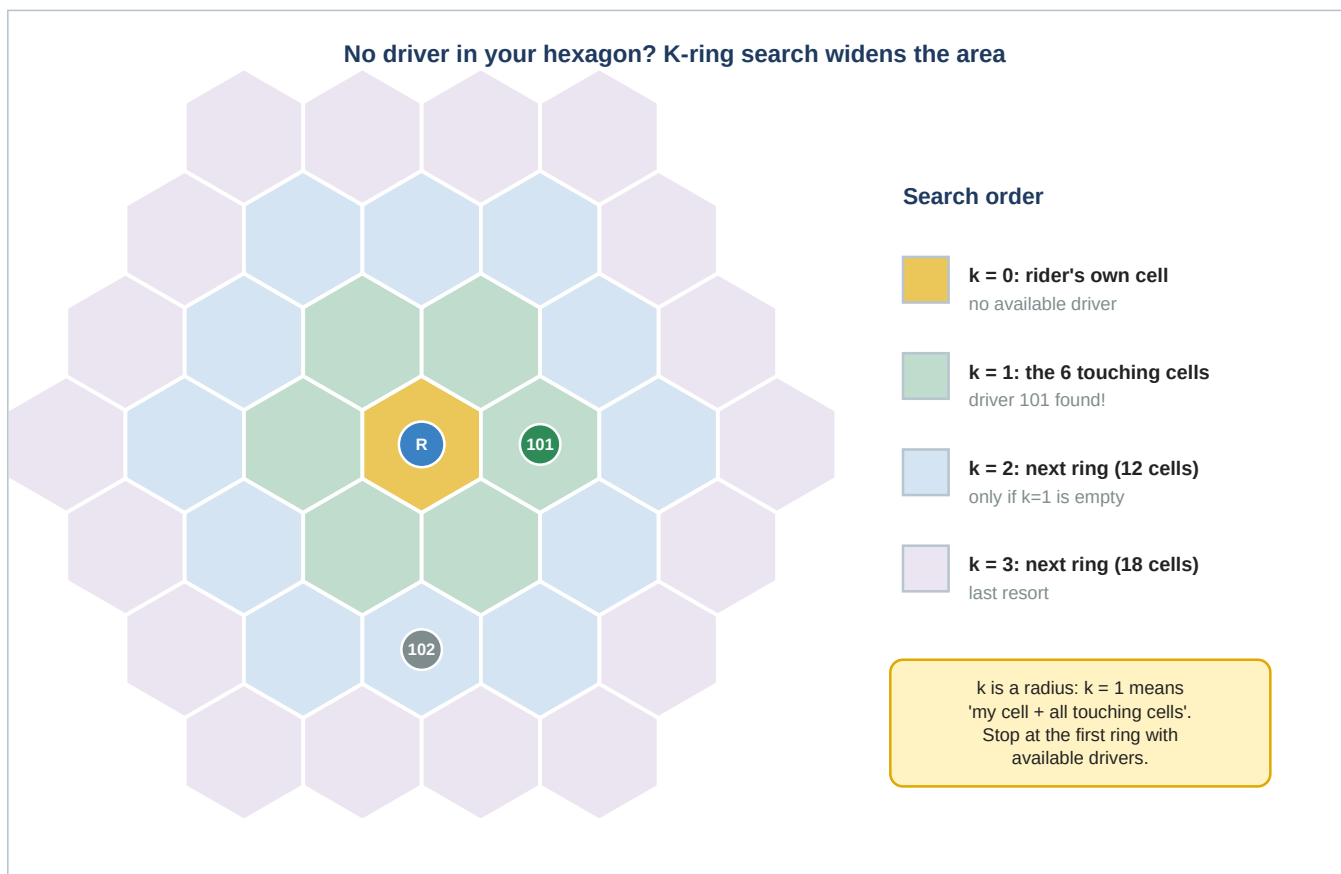


Figure 18: K-ring search widens the search area ring by ring

EXAMPLE

Rider R's cell has no free driver. Ring 1 (the 6 touching cells) contains driver 101, so 101 is a candidate. Driver 102 sits in ring 2 and is only considered if ring 1 had been empty. (The video describes K = 3 loosely as 'three adjacent hexagons'; technically K is a radius in rings.)

KEY TAKEAWAY

H3 turns 'find nearby drivers' into 'look up a cell ID', and K-ring turns 'nobody here' into 'look at the neighbours'.

6. Layer 4: The DISCO Matching Engine

The matching engine is the **brain** of the architecture. **DISCO** stands for **Dispatch Optimization** and is Uber's standard engine for ride matching.

6.1 Batch matching

Uber does **not** match one rider with one driver at a time. It **pools a bunch of requests** in a **2 to 5 second batch window**, gathers all the riders and all the drivers in the area, and solves a **bipartite matching problem** (riders on one side, drivers on the other) to find the best overall assignment.

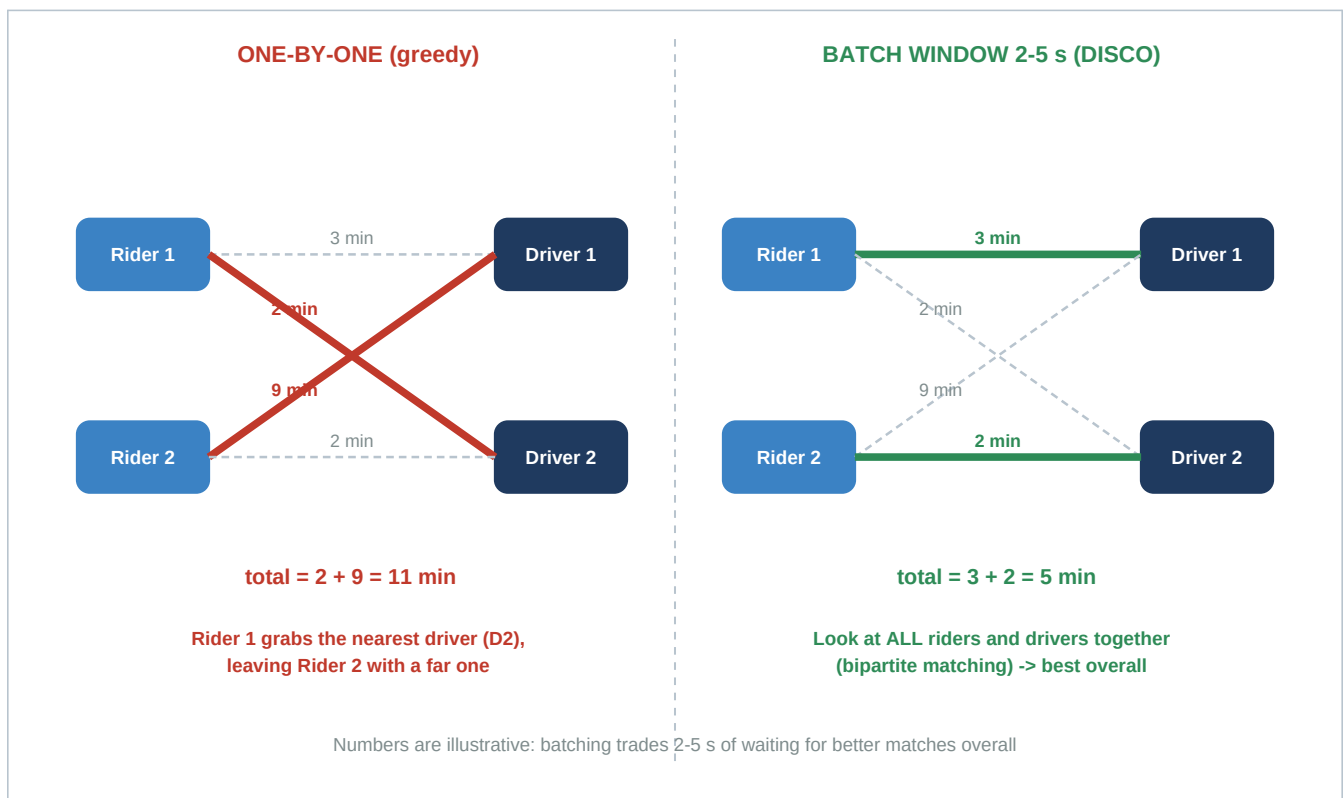


Figure 19: Greedy one-by-one matching vs batch matching

EXAMPLE

Illustrative numbers: Rider 1 is 3 min from Driver 1 and 2 min from Driver 2. Rider 2 is 9 min from Driver 1 and 2 min from Driver 2. Greedy: Rider 1 takes Driver 2 (2 min), forcing Rider 2 onto Driver 1 (9 min): total 11 min. Batch: Rider 1 -> Driver 1 (3), Rider 2 -> Driver 2 (2): total 5 min. A few seconds of waiting gives a much better overall result.

6.2 Deep ETA

Deep ETA is Uber's **deep learning model** that computes the **weight of the connection** between a rider and a driver (how long the pickup will take). It factors in **traffic, weather and even the time to find the rider**.

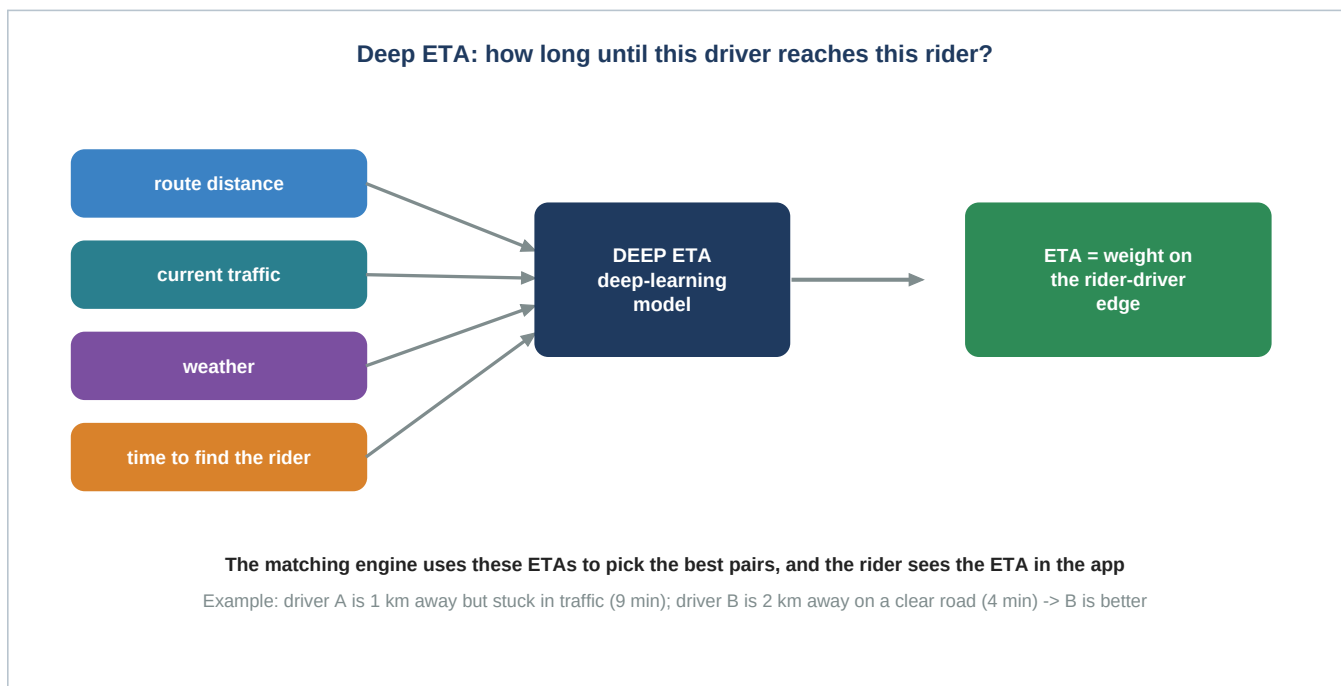


Figure 20: Deep ETA produces the edge weights used by the matcher

EXAMPLE

Driver A is 1 km away but on a jammed road (9 min). Driver B is 2 km away on a clear road (4 min). Straight-line distance would choose A; Deep ETA correctly prefers B and shows the rider '4 min'.

6.3 Atomic reserve (distributed locking)

The system is distributed and event-driven, so **several matching nodes work in parallel**. Without protection two nodes could assign the **same driver to two different riders**. **Atomic reserve** is a **distributed lock**, typically implemented in **Redis**, that **reserves a driver for about 10 seconds** while they decide whether to accept.

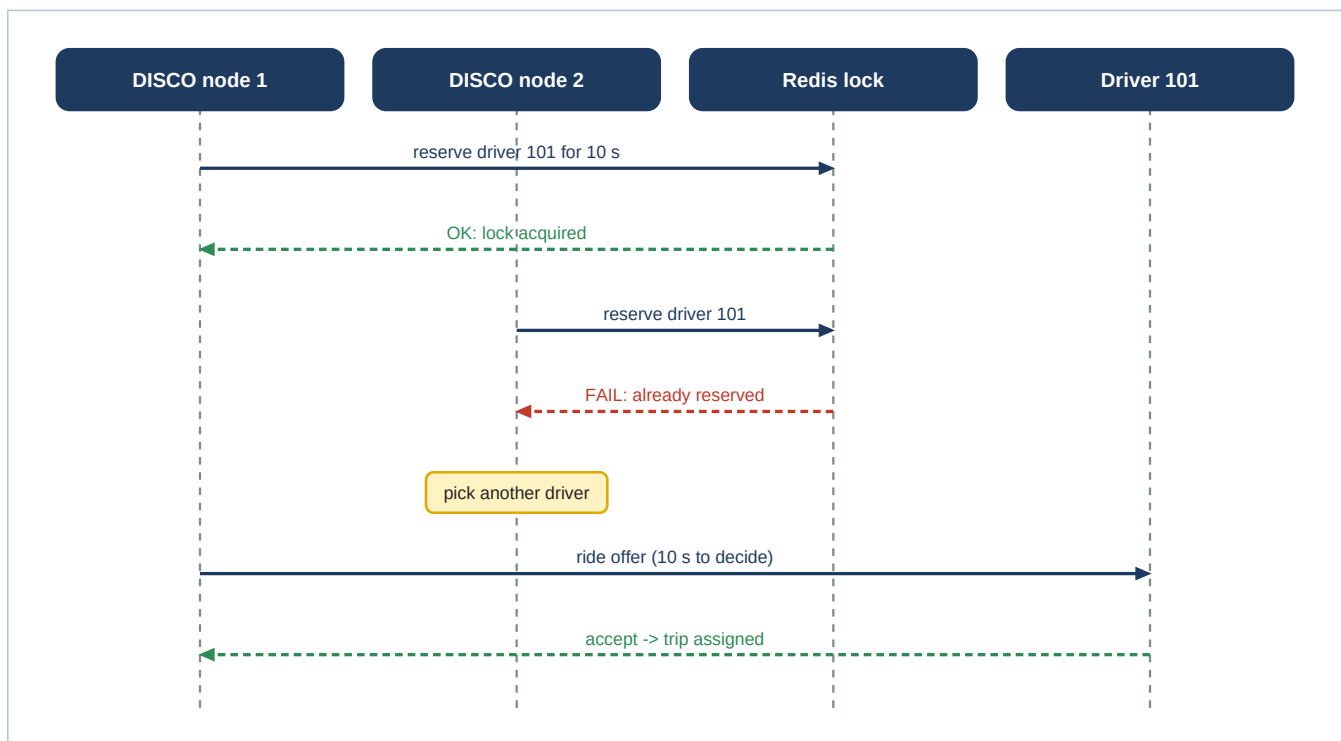


Figure 21: Only one matcher can reserve driver 101

EXAMPLE

Matcher 1 and Matcher 2 both want driver 101 at the same instant. The lock is atomic, so exactly one wins: Matcher 1 reserves 101 for 10 s and sends the offer. Matcher 2 is told 'already reserved' and picks another driver. If 101 ignores the offer, the reservation expires after 10 s and 101 becomes matchable again.

KEY TAKEAWAY

DISCO = batch matching + Deep ETA + atomic reserve. (The video notes distributed locks deserve their own video.)

7. Layer 5: Storage and Persistence

This layer is how the system remembers the **source of truth**.

7.1 Trip service: a state machine

The trip service manages the **life cycle of a trip**: **requested, assigned, started, completed**. Each state change is a critical event.

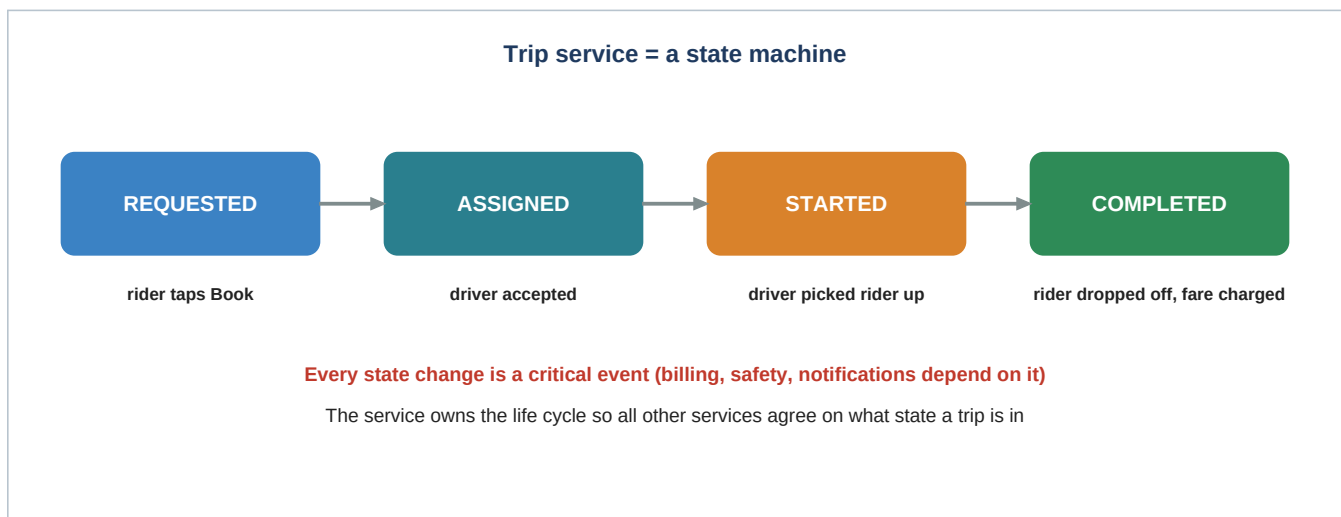


Figure 22: The trip life cycle

EXAMPLE

Requested: rider taps Book. Assigned: driver 101 accepted. Started: driver picked up the rider. Completed: drop-off, fare computed and charged. Billing, notifications and safety features all key off these transitions.

7.2 MySQL for payments and accounts

Uber uses a **heavily customised MySQL** (often referred to as the 'schemaless' layer) for **user profiles, payment methods, credit card tokens and drivers' bank details**. These need **absolute accuracy and ACID compliance**.

KEY TAKEAWAY

Wherever payments are involved, enforce ACID and use a SQL database.

7.3 Cassandra for trips

Uber uses **Cassandra** (NoSQL) for **trip details**, including the route taken, plus long-term trip and billing history. Reasons: **high availability** and the ability to handle **massive write volume across geographic locations**. Data is kept here for the first 7 days (the dispute window) and then moved to archive storage, although Cassandra could store data for years.

Two databases, chosen by the kind of data

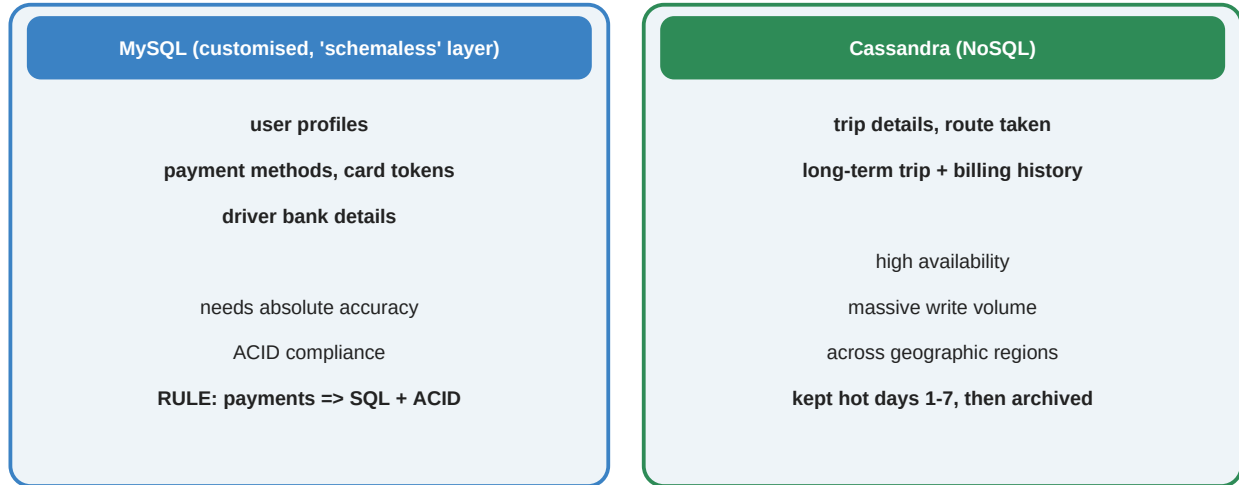


Figure 23: Which kind of data goes to which database

EXAMPLE

Ravi's bank account number and Asha's card token go to MySQL (one wrong digit is unacceptable, transactions must be atomic). The GPS route of Asha's trip, with thousands of points, goes to Cassandra (huge write volume, spread over regions, no complex joins).

8. End-to-End Walkthrough

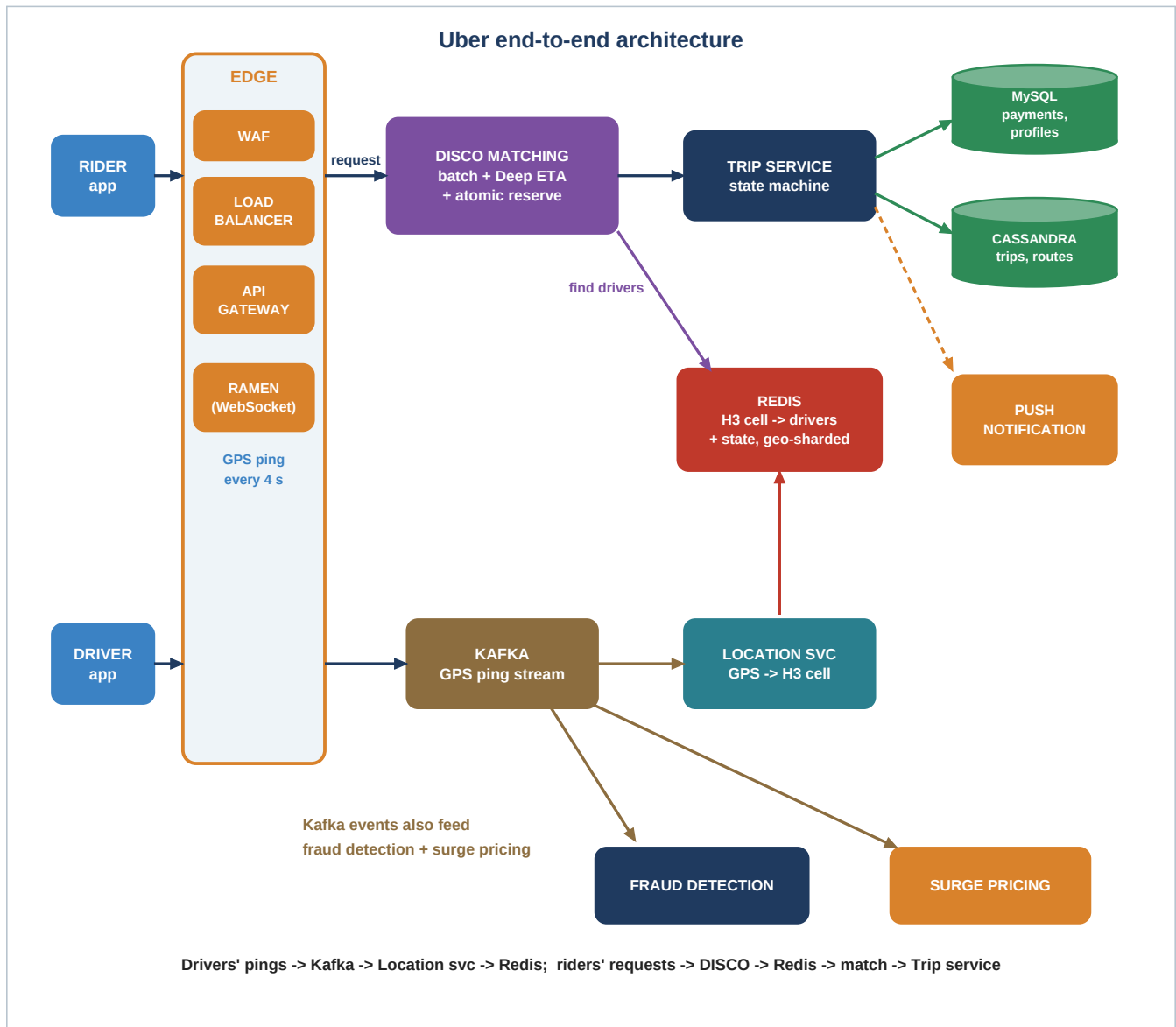


Figure 24: The complete Uber architecture

8.1 Driver side: location pipeline

- The **driver app** sends a ping every **4 seconds** through the **edge** (WAF, load balancer) over the persistent **Ramen** connection.
- Pings are written to **Kafka**.
- The **location service** reads pings from Kafka and converts GPS into an **H3 cell ID**.
- The cell ID is stored in **Redis** with the driver's status (available / on trip).

8.2 Rider side: matching

- The **rider app** sends a ride request through the edge (fraud and authentication checks, firewall, load balancing) over the Ramen connection.
- The request reaches **DISCO**, which **batches** requests.

- DISCO finds the rider's H3 cell and the **available drivers** in it (from Redis), using K-ring if needed.
- It matches riders to drivers, calling **Deep ETA** for the pickup times, and **atomically reserves** the chosen driver.
- The rider is told 'matched' with the ETA; the driver is notified via the **push notification service**.

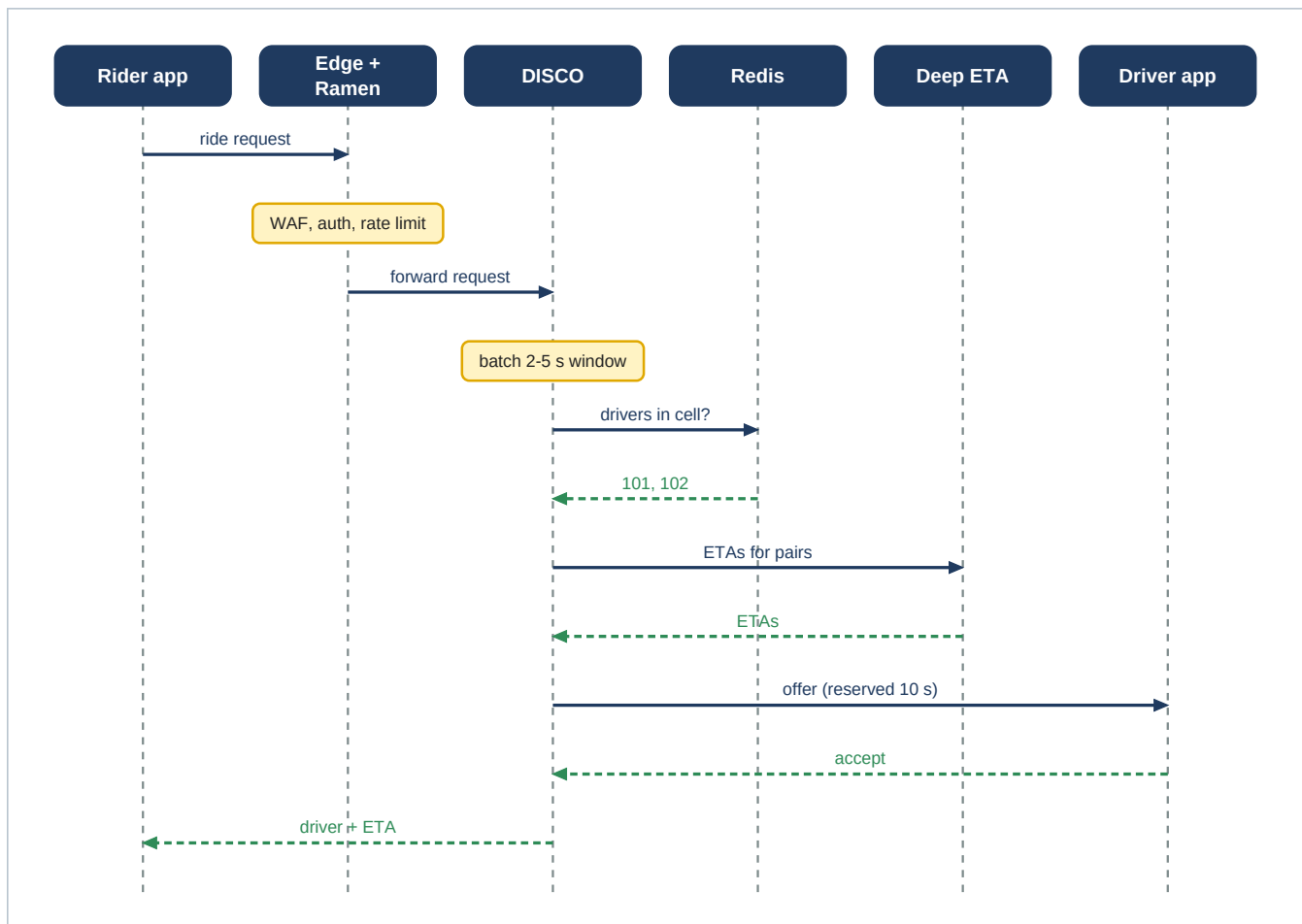


Figure 25: End-to-end sequence for one ride request

8.3 While the trip runs

- As the driver moves, new pings flow Kafka -> location service -> Redis; the driver hops between cells while marked on trip.
- The **trip service** moves the trip through its states and records data in **Cassandra** (trip details) and **MySQL** (payments, profiles).
- An independent **push notification service**, driven by DISCO, sends messages like 'your driver is arriving'.

8.4 Other services reading the Kafka stream

- **Fraud detection** uses the location events to spot suspicious behaviour (for example a driver whose GPS jumps impossibly).
- **Surge pricing** compares supply and demand per area: when drivers are few and riders many, prices rise (2x or 3x).

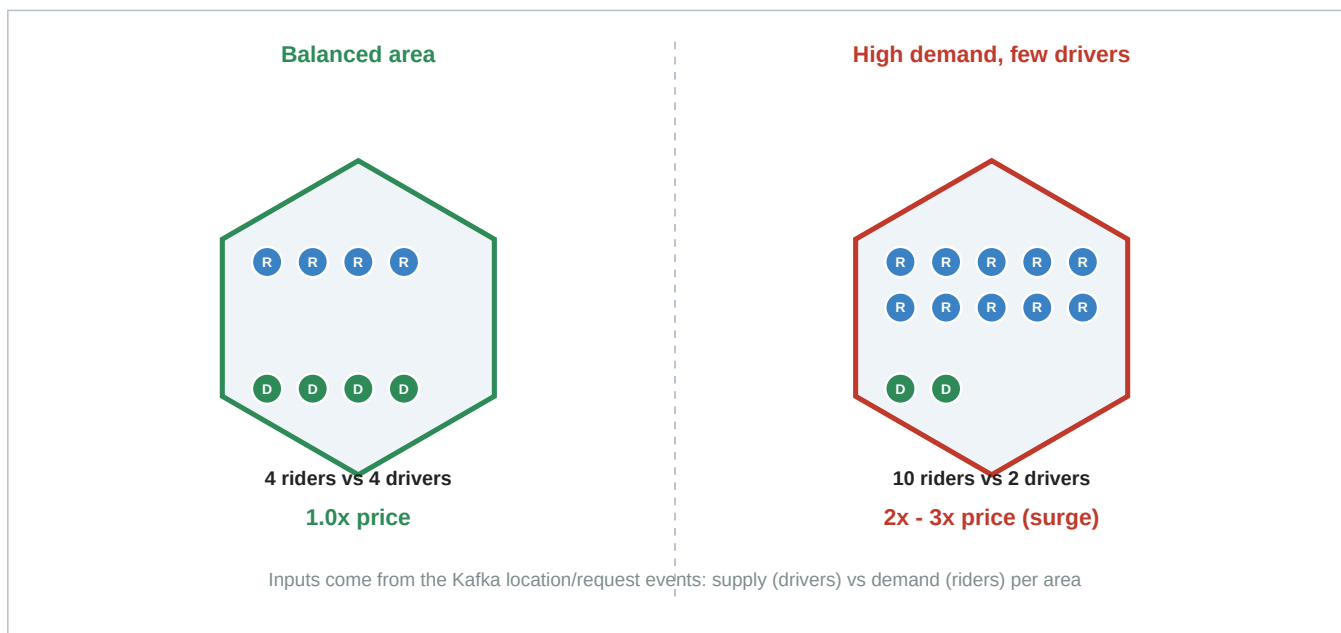


Figure 26: Surge pricing responds to demand exceeding supply in an area

EXAMPLE

After a concert, 10 riders in one hexagon compete for 2 drivers: surge pricing raises the multiplier to roughly 2x-3x, which encourages more drivers to head there and makes some riders wait. In a balanced area the price stays at 1x.

KEY TAKEAWAY

Interview tip from the video: explain the layers in order (edge, location pulse, H3 index, DISCO, storage) and you cover the whole architecture.

9. Cheat Sheet and Interview Q&A

9.1 One line per topic

Topic	Remember this
Capacity	~40 M trips/day = ~460 req/s; 2 M drivers x 1 ping/4 s = 500 K writes/s; 40 B per ping.
Storage	Trips 40 GB/day (14.6 TB/yr); pings 12 B/day = 480 GB/day (~175 TB/yr).
Hot/cold data	Cassandra for days 1-7 (dispute window), then S3/Hadoop archive.
Ramen	Uber's persistent push connection (WebSocket-style) for GPS up and trip updates down.
API gateway	Auth, rate limiting, routing to microservices; with WAF and load balancer.
Kafka	Durable stream of all pings; consumers: location svc, fraud, surge.
Redis	Latest location + state (available / on trip), geo-sharded by city.
H3	Hexagonal global grid; location -> 64-bit cell ID; 6 equidistant neighbours.
Cell lookup	Find drivers by cell ID instead of scanning all drivers.
K-ring	No driver in the cell: widen to neighbouring rings.
DISCO	Batch (2-5 s) bipartite matching + Deep ETA + atomic reserve.
Atomic reserve	Distributed lock (Redis) reserves a driver ~10 s: no double booking.
Trip service	State machine: requested, assigned, started, completed.
MySQL vs Cassandra	MySQL: payments, profiles (ACID). Cassandra: trips, routes (availability, huge writes).
Surge	Demand > supply in an area raises price (2x-3x).

9.2 Interview Q&A

Q: Describe the ride booking life cycle.

A: Rider requests, backend finds nearby drivers, the matcher picks the best, driver is notified, accepts, ride is confirmed.

Q: What is the heaviest load in Uber's design?

A: Location updates: 2 M drivers pinging every 4 s = 500 K writes/s.

Q: How big is one ping?

A: 40 bytes: driver ID, latitude, longitude, timestamp, metadata (8 B each).

Q: How much storage per day?

A: 40 GB of trip records and about 480 GB of location pings.

Q: What is Ramen?

A: Uber's persistent push platform (WebSocket-style) that streams GPS pings and trip updates in real time.

Q: Why Kafka for pings?

A: Durability: pings are not lost if matching is busy, and many services can consume the stream.

Q: Why Redis for locations?

A: In-memory fast lookup of every driver's latest location and state, geo-sharded by city.

Q: What is H3 and why hexagons?

A: Uber's hexagonal hierarchical spatial index; hexagons have six equidistant neighbours giving cleaner proximity search.

Q: How do you find nearby drivers fast?

A: Convert the rider's location to a cell ID and look up the available drivers in that cell.

Q: What if there is no driver in the cell?

A: K-ring search into neighbouring hexagon rings.

Q: What happens to the driver's record when they move?

A: It is removed from the old cell and added to the new one with the current status.

Q: Why batch requests for 2-5 seconds?

A: To solve a bipartite matching over many riders and drivers, giving a better overall result than greedy one-by-one.

Q: What does Deep ETA do?

A: A deep learning model estimating pickup time using traffic, weather and time to find the rider; it provides the match weights.

Q: How is double booking prevented?

A: Atomic reserve: a distributed lock (typically in Redis) reserves a driver for ~10 s.

Q: Which database for payments and why?

A: MySQL (SQL) because payments require absolute accuracy and ACID.

Q: Why Cassandra for trips and how long?

A: High availability and massive multi-region writes; trips stay hot for 7 days then move to S3/Hadoop archive.

Q: How does surge pricing work?

A: It reads supply/demand from the Kafka location stream per area and raises prices when drivers are scarce.

9.3 Notes on the source material

- The transcript is auto-generated, so a few terms are garbled: 'radius' = Redis, 'ramen' = Ramen (Uber's real-time messaging platform), and 'S3 ID / HID' = H3 cell ID. H3 here is Uber's hexagon index, not Amazon S3. Google's S2 library is the separate square-based cell index the speaker also alludes to.
- The speaker says 'we need to store 40 GB per day' right after computing 480 GB per day for pings; 480 GB/day (about 175 TB/year) is the correct figure, and 40 GB/day (14.6 TB/year) is the trip-record figure.
- Bandwidth units: the video says 20 Mbps and 100 Mbps; $500,000 \times 40$ bytes is 20 MB/s (160 Mbps) and the 5x egress is 100 MB/s (800 Mbps).
- The average ride requests per second is left unstated in the video; $40 \text{ M} / 86,400$ is about 460.
- K-ring: the video says $K = 3$ means 'three adjacent hexagons'; technically K is the number of rings searched outward.
- The batching numbers in this document's example and the cell ID shown are illustrative, not from the video. User counts (200 M MAU, 40 M DAU, 2 M drivers) are the speaker's assumptions.
- Atomic reserve and Deep ETA are described at a high level in the video; the video says distributed locks will be covered separately.